La lista doblemente enlazada: SplDoublyLinkedList

Caso práctico

María está programando el reproductor de la radio del instituto: canciones que se añaden al final, avisos urgentes que se ponen delante de todo, y botones **Siguiente** y **Anterior**. Una pila o una cola se le quedan cortas, porque necesita trabajar **por los dos extremos** y **moverse en los dos sentidos**. **Juan** le recomienda `SplDoublyLinkedList`, la clase en la que se basan `SplStack` y `SplQueue`.

1.- Qué es una lista doblemente enlazada

Una **lista enlazada** guarda los elementos en **nodos** repartidos por la memoria; cada nodo contiene un valor y un **enlace** al siguiente. En una lista **doblemente** enlazada cada nodo tiene además un enlace al **anterior**. Gracias a eso:

- se puede **añadir y quitar por los dos extremos** de forma inmediata (O(1)),
- se puede **recorrer en los dos sentidos**,
- pero llegar a la posición *i* obliga a ir saltando de nodo en nodo desde un extremo (O(n)), a diferencia de un array.

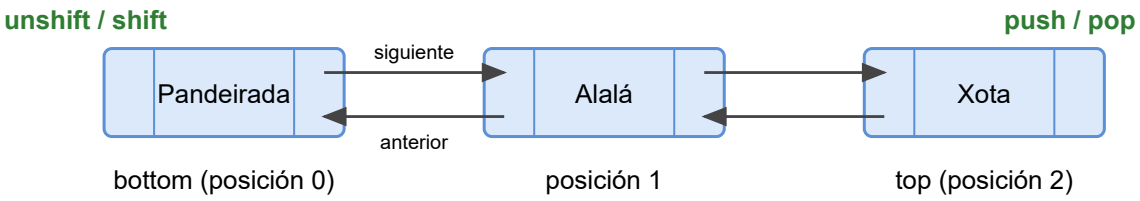


Figura 1. Una lista doblemente enlazada: cada nodo apunta al anterior y al siguiente

2.- La clase SplDoublyLinkedList

Operación	Método	Coste
Añadir al final / quitar del final	`push($x)` / `pop()`	O(1)
Añadir al principio / quitar del principio	`unshift($x)` / `shift()`	O(1)
Ver el último / el primero	`top()` / `bottom()`	O(1)
Insertar delante de la posición *i*	`add($i, $x)` (con `$i = count`, al final)	O(n)
Leer, cambiar, borrar la posición *i*	`$lista[$i]`, `$lista[$i] = $x`, `unset($lista[$i])`	O(n)
¿Existe la posición *i*?	`isset($lista[$i])`	O(n)
Tamaño / ¿vacía?	`count($lista)` / `isEmpty()`	O(1)

```php
<?php
$emisora = new SplDoublyLinkedList();
$emisora->push('Muiñeira de Chantada');
$emisora->push('Alalá');
$emisora->unshift('AVISO: tráfico en la AP-9');     // al principio
echo $emisora->bottom();                             // "AVISO: tráfico en la AP-9"
echo $emisora->shift();                              // lo saca
$emisora->add(1, 'Foliada');                         // delante de la posición 1
unset($emisora[0]);                                  // borra la posición 0 y reenumera
```

Debes conocer

- Las posiciones **se reenumeran** después de `add()` y de `unset()`: si borras dentro de un `foreach`, sal del bucle (o recorre la lista de nuevo), porque las posiciones han cambiado.
- Acceder a una posición que no existe lanza `OutOfRangeException`; sacar de una lista vacía, `RuntimeException`.
- Ninguna de estas clases tiene `toArray()`: usa `iterator_to_array($lista, false)`.

3.- Modos de recorrido

A diferencia de `SplStack` y `SplQueue`, en una `SplDoublyLinkedList` se puede elegir cómo la recorre `foreach` con `setIteratorMode()`, combinando con `|` una opción de cada grupo:

Constante	Significado
`IT_MODE_FIFO` (por defecto)	Del principio (*bottom*) al final (*top*)
`IT_MODE_LIFO`	Del final al principio
`IT_MODE_KEEP` (por defecto)	Los elementos se conservan
`IT_MODE_DELETE`	Cada elemento recorrido se elimina

```php
<?php
$pedidos->setIteratorMode(SplDoublyLinkedList::IT_MODE_LIFO);          // del más reciente al más antiguo
$pedidos->setIteratorMode(SplDoublyLinkedList::IT_MODE_FIFO | SplDoublyLinkedList::IT_MODE_DELETE);  // procesar y vaciar
```

Una `SplStack` es una lista en modo LIFO y una `SplQueue`, en modo FIFO, con ese sentido **bloqueado**.

Recorrido manual. La lista tiene un cursor interno que se maneja con `rewind()` (al principio), `next()`, `prev()`, `valid()`, `current()` y `key()`. Es lo que permite los botones "siguiente" y "anterior" de un reproductor:

```php
<?php
$playlist->rewind();
$playlist->next();
$playlist->next();
echo $playlist->current();      // la tercera canción
$playlist->prev();
echo $playlist->current();      // la segunda
```

4.- Aplicaciones típicas

Problema	Por qué una lista doblemente enlazada
Cola de doble extremo (*deque*): pedidos normales y urgentes	Se añade y se quita por los dos extremos en O(1)
Reproductores, carruseles, presentaciones	Se avanza y se retrocede (`next` / `prev`)
Historial con "atrás" y "adelante"	Se recorre en los dos sentidos y se recorta por el final
Cachés LRU (los usados más recientemente)	Mover un elemento al principio y descartar el del final
Comprobar palíndromos, ventanas deslizantes	Comparar o quitar por los dos extremos

5.- Ejercicio resuelto: caché de los productos más consultados

Ejercicio resuelto

La tienda guarda en memoria las fichas de los 3 productos consultados más recientemente. Si se consulta uno que no está y la caché está llena, se descarta el que hace más tiempo que no se usa (*LRU, Least Recently Used*).

```php
<?php
final class CacheLru
{
    private SplDoublyLinkedList $orden;      // claves, de la más reciente a la más
```

```
antigua
    private array $datos = [];

    public function __construct(private int $capacidad)
    {
        $this->orden = new SplDoublyLinkedList();
    }

    public function obtener(string $clave, callable $cargar): mixed
    {
        if (array_key_exists($clave, $this->datos)) {
            $this->quitarDelOrden($clave);                // acierto: se moverá al
principio
        } else {
            if (count($this->orden) === $this->capacidad) {
                unset($this->datos[$this->orden->pop()]);  // fuera el usado hace más
tiempo
            }
            $this->datos[$clave] = $cargar($clave);       // fallo: se carga (de la base
de datos...)
        }
        $this->orden->unshift($clave);                    // ahora es el más reciente
        return $this->datos[$clave];
    }

    private function quitarDelOrden(string $clave): void
    {
        foreach ($this->orden as $i => $c) {
            if ($c === $clave) {
                $this->orden->offsetUnset($i);
                return;                                   // las posiciones han cambiado:
salimos
            }
        }
    }
}
$cache = new CacheLru(3);
foreach (['RAT', 'TEC', 'MON', 'RAT', 'CAS', 'TEC'] as $ref) {
    $cache->obtener($ref, fn($r) => "ficha de $r");
}
// Al final la caché contiene, de más reciente a más antiguo: TEC, CAS, RAT
// (al llegar CAS se descartó TEC, y al volver TEC se descartó MON)
```

Observa que la lista solo guarda el **orden** de uso, y el array, los **datos**. Una caché LRU profesional evita también el recorrido de `quitarDelOrden()` guardando para cada clave una referencia a su nodo, pero la idea es la misma.

Autoevaluación

- ¿Qué métodos trabajan por el principio de la lista? **`unshift()`, `shift()` y `bottom()`.**
- ¿Por qué acceder a `$lista[5000]` es más lento que en un array? **Porque hay que recorrer los nodos uno a uno desde un extremo (O(n)).**
- ¿Cómo se recorre una `SplDoublyLinkedList` del final al principio? **Con `setIteratorMode(SplDoublyLinkedList::IT_MODE_LIFO)`.**
- ¿Qué relación hay entre `SplDoublyLinkedList`, `SplStack` y `SplQueue`? **Las dos últimas heredan de la primera, con el modo de recorrido fijado (LIFO y FIFO).**

6.- Ejercicios

Fichero	Contenido
`Ejercicios DoublyLinkedList/e01-SplDoublyLinkedList.php`	La API: los dos extremos, `add` y posiciones, modos de recorrido, `next` / `prev`
`Ejercicios DoublyLinkedList/e02-SplDoublyLinkedList.php`	Aplicaciones: deque de pedidos, palíndromos, lista ordenada, caché LRU, lista circular
`Ejercicios DoublyLinkedList/problemas/`	`e01` y `e02`: problemas y soluciones